

type: Note cssclasses: - cornell-notes

Back to [[Pembelajaran Mesin IF3270]]

[!cornell] Perceptron Fundamentals and Biological Analogy

Questions/Cues

- Mengapa perceptron menggunakan fungsi aktivasi tangga?
- Bagaimana struktur perceptron meniru neuron biologis?
- Apa perbedaan input biner vs kontinu pada perceptron?
- Mengapa perceptron hanya bekerja untuk data linearly separable?
- Bagaimana merepresentasikan fungsi logika AND/OR dengan perceptron?

Reference Points

- Lecture_01_Perceptron.pptx (Slides 5-12, 17)
- Machine_Learning_Raschka_2022 (Hal 32-35)

Konsep Biologis Neuron

Perceptron terinspirasi dari model biologis neuron yang diusulkan McCulloch dan Pitts (1942). Neuron biologis menerima sinyal kimia dan elektrik melalui dendrit, mengintegrasikannya di badan sel, dan menghasilkan sinyal output melalui akson jika ambang batas tertentu terlampaui. Analogi dengan perceptron: 1. **Input ($x_1, x_2, \dots, x_n$)** $\equiv$ Sinyal dendrit 2. **Bobot ($w_1, w_2, \dots, w_n$)** $\equiv$ Kekuatan sinapsis 3. **Fungsi Aktivasi** $\equiv$ Proses integrasi badan sel 4. **Output (y)** $\equiv$ Sinyal akson Contoh konkret: Saat menerima input penting (bobot tinggi) dan input sekunder (bobot rendah), perceptron mengkombinasikannya secara linear sebelum menerapkan fungsi aktivasi tangga.

Definisi Formal Perceptron

Perceptron adalah model jaringan saraf tiruan paling sederhana untuk klasifikasi biner. Secara matematis didefinisikan sebagai: $y = \text{sign}(\sum w_i x_i + b) = \text{sign}(w^T x + b)$ - **x** : Vektor input fitur ($x_1, x_2, \dots, x_n$) - **w** : Vektor bobot (parameter yang dipelajari) - **b** : Bias (threshold) - **sign**: Fungsi aktivasi tangga (output +1 atau -1) Model ini membentuk **hyperplane keputusan** di ruang n-dimensi. Untuk kasus 2D, ini

berupa garis lurus yang memisahkan dua kelas. Contoh visual: Pada slide 8, garis $0.5x_1 + 0.5x_2 = 0.8$ memisahkan titik AND (+1) dari lainnya (-1).

Representasi Fungsi Logika

Perceptron dapat merepresentasikan fungsi logika dasar dengan konfigurasi bobot spesifik: **Fungsi AND**: - Bobot: $w_0 = -0.8$, $w_1 = 0.5$, $w_2 = 0.5$ - Persamaan: $0.5x_1 + 0.5x_2 \geq 0.8 \rightarrow y=+1$ **Fungsi OR**: - Bobot: $w_0 = -0.5$, $w_1 = 1.0$, $w_2 = 1.0$ - Persamaan: $1.0x_1 + 1.0x_2 \geq 0.5 \rightarrow y=+1$ Pola umum untuk **fungsi m-of-n**: 1. Set semua bobot input sama (misal 0.5) 2. Atur bias (w_0) sesuai kebutuhan m Contoh: Fungsi 2-of-3 memerlukan $w_0 = -1.0$ dengan bobot input seragam 0.5.

Batasan dan Linear Separability

Perceptron hanya berfungsi optimal untuk data **linearly separable**—kelas dapat dipisahkan oleh hyperplane linier. Keterbatasan utama: 1. Tidak bisa menyelesaikan masalah non-linear seperti XOR 2. Sensitif terhadap noise dan outlier 3. Membutuhkan learning rate tepat untuk konvergensi Contoh kasus gagal: Data XOR (Slide 10) tidak linearly separable dalam ruang 2D. Solusi memerlukan transformasi fitur atau model lebih kompleks (misal multilayer perceptron).

[!cornell] ##### Summary

Perceptron adalah model jaringan saraf tiruan paling dasar yang terinspirasi dari neuron biologis, menggunakan **fungsi aktivasi tangga** untuk klasifikasi biner. Model ini efektif untuk masalah **linearly separable** seperti implementasi fungsi logika AND/OR melalui konfigurasi bobot spesifik. **Batasan utama** terletak pada ketidakmampuan menangani hubungan non-linear antar fitur, memerlukan teknik lanjutan untuk kasus lebih kompleks.

[!ad-libitum]- Additional Information

Analisis Matematis Fungsi Aktivasi

Fungsi signum (sign) dapat dinyatakan secara formal sebagai: $\text{sign}(z) = \{ +1 \text{ jika } z \geq 0, -1 \text{ jika } z < 0 \}$ Derivatif fungsi ini adalah nol di semua titik kecuali $z=0$ (tidak terdiferensiasi), membatasi penggunaan metode berbasis gradien. Pada implementasi praktis, sering digunakan aproksimasi seperti fungsi sigmoid atau ReLU untuk keperluan diferensiasi.

Implementasi Praktis dengan Python

```
```python import numpy as np
```

```
class Perceptron:
 def init(self, input_size):
 self.weights = np.zeros(input_size+1) # +1 untuk bias
 def predict(self, x):
 z = np.dot(x, self.weights[1:]) + self.weights[0]
 return 1 if z >= 0 else -1
```

## Aplikasi dalam Computer Vision Modern

Meskipun sederhana, arsitektur perceptron menjadi dasar untuk:

- Convolutional Neural Networks (CNN) melalui konsep operasi konvolusi linier
- Attention mechanisms dalam transformer dengan proyeksi linier
- Embedding layers yang memetakan input ke ruang vektor

## Bacaan Lanjutan

- McCulloch, W.S. & Pitts, W. (1943). "A Logical Calculus of Ideas Immanent in Nervous Activity". Bulletin of Mathematical Biophysics
- Rosenblatt, F. (1958). "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain". Psychological Review
- Nielsen, M. (2019). "Neural Networks and Deep Learning". Chapter 1: Using Neural Nets to Recognize Handwriting

## Eksperimen Mandiri

1. Implementasikan perceptron dari nol untuk dataset iris (klasifikasi setosa vs non-setosa)
2. Visualisasi decision boundary menggunakan matplotlib untuk berbagai konfigurasi bobot
3. Eksplorasi efek learning rate terhadap kecepatan konvergensi

## Tools Rekomendasi

- `sklearn.linear_model.Perceptron` : Implementasi optimisasi perceptron
- TensorFlow Playground: Visualisasi interaktif (<https://playground.tensorflow.org>)
- Google Colab: Platform eksperimen berbasis cloud

---

type: Note cssclasses: - cornell-notes

---

Back to [[Pembelajaran Mesin IF3270]]

## [!cornell] Perceptron Learning Rule Algorithm

## Questions/Cues

- Mengapa perceptron memerlukan aturan pembelajaran spesifik?
- Bagaimana langkah pembaruan bobot bekerja per-contoh?
- Apa perbedaan esensial batch vs stochastic update?
- Kriteria penghentian pelatihan perceptron?
- Bagaimana learning rate memengaruhi konvergensi?

## Reference Points

- IF3270 Pembelajaran Mesin - Perceptron (Slides 14-18, 27-31)
- Raschka, Liu, & Mirjalili (2022) - Machine Learning with PyTorch and Scikit-Learn (Slide 3)

## Mekanisme Pembelajaran Perceptron

Algoritma pembelajaran perceptron bertujuan menemukan vektor bobot optimal yang memisahkan kelas positif dan negatif secara linear. Proses dimulai dengan inisialisasi bobot awal secara acak (biasanya nilai kecil mendekati nol). Untuk setiap contoh pelatihan, model membuat prediksi menggunakan fungsi aktivasi signum yang menghasilkan output +1 atau -1. Jika prediksi salah, dilakukan penyesuaian bobot dengan rumus:  $\mathbf{w}_j = \mathbf{w}_j + \Delta \mathbf{w}_j$  dimana  $\Delta \mathbf{w}_j = \eta(\mathbf{y}_i - \mathbf{o}_i)\mathbf{x}_{ij}$ .  $\eta$  (learning rate) mengontrol besarnya penyesuaian,  $\mathbf{y}_i$  adalah label sebenarnya,  $\mathbf{o}_i$  prediksi model, dan  $\mathbf{x}_{ij}$  nilai fitur. Contoh: Jika fitur  $\mathbf{x}_{ij}$  bernilai tinggi dan terjadi kesalahan klasifikasi, penyesuaian bobot akan lebih signifikan dibandingkan fitur dengan nilai rendah.

## Proses Pembaruan Incremental

Berbeda dengan metode batch yang menghitung gradien seluruh dataset, perceptron umumnya menggunakan pembaruan stochastic per-contoh. Setiap kesalahan klasifikasi langsung memicu penyesuaian bobot, membuat algoritma lebih responsif terhadap pola data baru. Pendekatan ini efisien untuk dataset besar karena tidak perlu menyimpan seluruh gradien dalam memori. Implementasi tipikal menggunakan loop berulang (epoch) melalui dataset sampai tidak ada kesalahan klasifikasi atau mencapai batas iterasi maksimum. Pada setiap epoch, contoh diacak untuk mencegah pola pembelajaran berurutan yang bias.

## Pengaruh Learning Rate

Learning rate ( $\eta$ ) menentukan kecepatan pembelajaran: nilai terlalu tinggi menyebabkan overshooting minimum global, sementara nilai terlalu rendah memperlambat konvergensi. Nilai optimal biasanya di range 0.1 sampai 0.0001. Contoh praktis: Untuk dataset dengan fitur ternormalisasi,  $\eta=0.1$  umumnya efektif. Pada slide 16 ditunjukkan contoh nyata dengan  $\eta=0.1$  yang berhasil mengkonvergensi bobot dalam dua epoch untuk fungsi AND sederhana.

## Kriteria Penghentian

Pelatihan berhenti ketika: (1) Semua contoh terklasifikasi benar (error=0), (2) Mencapai batas maksimum epoch, atau (3) Error rate stabil meskipun iterasi berlanjut. Penting untuk memasukkan batas iterasi maksimum karena perceptron tidak konvergen untuk data tidak terpisahkan linear (non-linearly separable). Contoh pada slide 18 menunjukkan terminasi ketika error mencapai nol setelah beberapa iterasi.

[!cornell] ##### Summary **Aturan pembelajaran perceptron** menggunakan pendekatan pembaruan bobot incremental berdasarkan kesalahan klasifikasi per-contoh dengan rumus  $\Delta w_j = \eta(y_i - o_i)x_{ij}$ . Algoritma ini **efisien untuk data terpisah linear** namun memerlukan penanganan khusus untuk kasus non-linear. **Learning rate** menentukan kecepatan dan stabilitas konvergensi, sementara kriteria penghentian harus mempertimbangkan **kemungkinan non-konvergensi**. Implementasi praktis memanfaatkan pembaruan stokastik dan pengacakan data tiap epoch.

[!ad-libitum]- Additional Information

## Analisis Konvergensi

Teori konvergensi perceptron menjamin solusi dalam waktu terbatas jika data linearly separable. Untuk dimensi  $d$  dan margin  $\gamma$ , maksimum iterasi yang dibutuhkan dibatasi oleh  $(R/\gamma)^2$  dimana  $R$  adalah radius bola mencakup semua titik data. Bukti formal menggunakan pertidaksamaan Cauchy-Schwarz menunjukkan penurunan bound kesalahan kuadratik setiap iterasi.

## Perbandingan Batch vs Stochastic

Pembaruan batch (slide 23) menghitung gradien rata-rata seluruh dataset sebelum update bobot, menjamin penurunan error monoton tapi komputasinya berat. Stochastic update (slide 29) lebih cepat per iterasi tapi berosilasi mendekati

minimum. Komprominya: mini-batch (slide 31) menggabungkan kelebihan kedua pendekatan dengan ukuran batch tipikal 32-512 contoh.

### Eksperimen Parameter Learning Rate

Lakukan simulasi dengan  $\eta$  berbeda (0.001, 0.01, 0.1, 1) pada dataset XOR (non-linear) dan AND (linear). Amati: (1) Jumlah epoch sampai konvergen (jika mungkin), (2) Perilaku error function, (3) Pemisahan decision boundary yang dihasilkan. Hasil akan menunjukkan ketidakmampuan perceptron menangani XOR dan sensitivitas terhadap pemilihan  $\eta$ .

### Implementasi dengan Python

```
python import numpy as np class Perceptron: def __init__(self, lr=0.01, epochs=1000): self.lr = lr self.epochs = epochs def fit(self, X, y): self.weights = np.zeros(X.shape[1]) for _ in range(self.epochs): for xi, yi in zip(X, y): pred = np.sign(np.dot(xi, self.weights)) if pred != yi: update = self.lr * (yi - pred) self.weights += update * xi
```

### Further Reading

- "Perceptron-Based Learning Algorithms" (IEEE Transactions on Neural Networks, 1990)
- "The Convergence Proof for the Perceptron Algorithm" (Block & Novikoff, 1962)
- Scikit-learn Documentation: Perceptron Implementation
- Interactive Visualization: <https://playground.tensorflow.org>
- Coursera: "Neural Networks and Deep Learning" (Week 1)

type: Note cssclasses: - cornell-notes

Back to [[Pembelajaran Mesin IF3270]]

[!cornell] Batch Gradient Descent Optimization

## Questions/Cues

- Mengapa Batch GD menggunakan seluruh dataset?
- Bagaimana mekanisme pembaruan bobot Batch GD?

- Perbedaan konsep epoch antara Batch dan Stochastic GD
- Keuntungan Batch GD pada error surface konveks
- Tantangan konvergensi pada data non-separable

## Reference Points

- Perceptron IF3270 (Slides 19-26, 28-31)
- Raschka (2022) Chapter on Optimization (Hal 142-150)

## Pengertian Batch Gradient Descent

Batch Gradient Descent (BGD) adalah algoritma optimasi yang memperbarui parameter model dengan menghitung gradien error menggunakan seluruh dataset pelatihan pada setiap iterasi. Berbeda dengan pendekatan inkremental, BGD mengakumulasi informasi dari semua sampel sebelum melakukan pembaruan bobot. Prinsip dasarnya adalah meminimalkan fungsi error (biasanya Mean Squared Error) dengan bergerak berlawanan arah gradien. Jika diibaratkan mencari lembah terendah, BGD menghitung kemiringan seluruh permukaan bukit sebelum memutuskan langkah berikutnya. Ini menjadikannya lebih stabil namun komputasional lebih berat dibanding varian stokastik. Contoh penerapannya pada perceptron: Misal terdapat 100 sampel latih, BGD akan menghitung akumulasi error dari seluruh 100 sampel tersebut sebelum menyesuaikan bobot sekali saja. Pendekatan ini menjamin penurunan error konsisten ke arah gradien global.

## Mekanisme Pembaruan Bobot

Proses pembaruan bobot dalam BGD mengikuti rumus:  $\Delta w = \eta * (1/m) * \sum (y_i - \hat{y}_i) * x_i$  di mana: -  $\eta$ : Learning rate (0.1 pada contoh slide 23) -  $m$ : Jumlah sampel dalam dataset -  $y_i$ : Label sebenarnya -  $\hat{y}_i$ : Prediksi model (output fungsi aktivasi) -  $x_i$ : Vektor fitur Pada implementasinya (Slide 23): 1. Inisialisasi bobot dengan nilai kecil acak ( $w = [0,0,0,0]$ ) 2. Untuk setiap epoch: - Hitung prediksi semua sampel ( $o_i$ ) - Akumulasi  $\Delta w$  dari seluruh sampel - Pembaruan bobot hanya dilakukan sekali per epoch 3. Terminasi saat error konvergen Contoh numerik dari slide: Dengan learning rate 0.3, bobot diperbarui setelah memproses semua sampel, menghasilkan perubahan lebih stabil dibanding pembaruan per-sampel.

## Perbandingan dengan Stochastic Gradient Descent

Perbedaan mendasar terletak pada frekuensi pembaruan bobot (Slide 28): - **Batch GD**: Pembaruan 1 kali per epoch (setelah akumulasi seluruh sampel) - **Stochastic GD**: Pembaruan tiap sampel ( $m$  kali per epoch) Keunggulan BGD: 1. Arah gradien lebih stabil karena menggunakan informasi komplet 2. Konvergensi lebih halus

pada error surface konveks (Gambar slide 26) 3. Deterministik - hasil reproduktif dengan inisialisasi sama Kelemahan BGD: 1. Komputasi berat untuk dataset besar 2. Risiko terjebak minimum lokal pada error surface kompleks 3. Sensitif terhadap learning rate yang tidak tepat

## Tantangan Implementasi Praktis

Masalah utama BGD (Slide 26): 1. **Konvergensi Lambat**: Diperlukan banyak epoch untuk mencapai minimum global, terutama jika learning rate kecil. Solusi: Line search adaptif. 2. **Minimum Lokal**: Pada fungsi error non-konveks, BGD tidak menjamin menemukan solusi optimal global. Teknik seperti momentum dapat membantu. 3. **Sensitivitas Learning Rate**:  $\eta$  terlalu besar menyebabkan osilasi; terlalu kecil memperlambat konvergensi. Praktik terbaik: decay schedule ( $\eta = \eta_0 / (1 + \text{decay}t)$ ). 4. Non-Separabilitas Data\*: Jika data tidak linearly separable, BGD akan konvergen ke solusi "terbaik" namun tidak sempurna (Slide 20). Error residual tetap ada meski dengan epoch tak terbatas.

[!cornell] ##### Summary

**Batch Gradient Descent** adalah algoritma optimasi yang memperbarui parameter model berdasarkan gradien rata-rata dari seluruh dataset. Metode ini menjamin penurunan error konsisten namun memerlukan komputasi intensif. Keunggulan utamanya terletak pada **stabilitas konvergensi** untuk fungsi error konveks, sementara kelemahan utamanya adalah **kinerja buruk pada data berskala besar**. Implementasi praktis memerlukan pertimbangan matang terhadap **pemilihan learning rate** dan **mekanisme terminasi** untuk menghindari konvergensi prematur atau komputasi berlebihan.

[!ad-libitum]- Additional Information

## Analisis Matematis Formulasi BGD

Turunan parsial fungsi error  $E(w)$  untuk perceptron dengan aktivasi linear:  $\partial E / \partial w_j = (1/m) \sum (y_i - \hat{y}_i) * x_{ij}$  Pembaruan bobot mengikuti arah steepest descent:  $w^{(t+1)} = w^{(t)} - \eta \nabla E(w^{(t)})$  Konvergensi dijamin jika  $\eta < 2/\lambda_{\max}$  dengan  $\lambda_{\max}$  eigenvalue terbesar matriks kovarians fitur.

## Implementasi Numerik

Kode Python sederhana untuk BGD: 

```
python def batch_gd(X, y, lr=0.01, epochs=100): w = np.zeros(X.shape[1]) for _ in range(epochs): grad = np.dot(X.T, (y - np.dot(X, w))) / len(y) w += lr * grad return w
```



## Studi Kasus: Data Non-Linear

Pada dataset XOR (Slide 10), BGD gagal mencapai error nol karena keterbatasan linearitas perceptron. Error residual minimum yang dapat dicapai adalah 0.5 pada contoh tersebut, menunjukkan perlunya transformasi fitur atau model non-linear.

## Tools Visualisasi

1. TensorFlow Playground: Eksplorasi interaktif pengaruh learning rate
2. Matplotlib Animasi: Visualisasi lintasan konvergensi di ruang parameter
3. Plotly Surface Plot: Representasi 3D error surface

## Proyek Eksplorasi Mandiri

1. Implementasikan BGD untuk regresi linear dengan dataset Boston Housing. Bandingkan kecepatan konvergensi vs SGD.
2. Modifikasi algoritma dengan Nesterov Momentum dan analisis pengaruhnya terhadap kecepatan konvergensi.
3. Eksperimen dengan adaptive learning rate (Adam vs Adagrad) pada kasus data tidak terpisah linier.

## Bacaan Lanjutan

- "Convex Optimization" oleh Boyd & Vandenberghe (Bab 9: Unconstrained Minimization)
- "Neural Networks and Learning Machines" oleh Haykin (Bab 3: Gradient Descent)
- Paper: "Understanding Machine Learning: From Theory to Algorithms" (Shalev-Shwartz & Ben-David)
- Dokumentasi SciPy: Optimasi Non-Linier dengan Metode Quasi-Newton

---

type: Note cssclasses: - cornell-notes

---

Back to [[Pembelajaran Mesin IF3270]]

[!cornell] Stochastic Gradient Descent Variants

## Questions/Cues

- Mengapa SGD lebih efisien untuk data besar?
- Perbedaan utama Batch GD vs SGD?
- Cara kerja update weight pada SGD?
- Keuntungan menggunakan mini-batch?
- Tantangan implementasi SGD nyata?

## Reference Points

- Lecture\_IF3270\_Perceptron.pptx (Slides 27-31)
- Machine Learning with PyTorch and Scikit-Learn (Raschka et al., 2022)

## Konsep Dasar Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) merupakan varian optimasi yang memperbarui parameter model secara iteratif untuk setiap contoh pelatihan, berbeda dengan pendekatan batch yang menggunakan seluruh dataset sekaligus. Pada SGD, pembaruan bobot dilakukan setelah menghitung gradien untuk **setiap contoh individual**, membuatnya lebih efisien secara komputasi untuk dataset berskala besar. Analoginya seperti menuruni bukit dengan langkah kecil-kecil namun cepat, di mana setiap langkah hanya berdasarkan informasi lokal (satu titik data), bukan peta lengkap seluruh permukaan (seluruh dataset). Ini memungkinkan konvergensi lebih cepat pada dataset besar karena pembaruan lebih sering terjadi. Contoh: Pada dataset dengan 1 juta sampel, SGD melakukan 1 juta pembaruan per epoch, sedangkan Batch GD hanya 1 pembaruan.

## Implementasi dan Prosedur Pembelajaran SGD

Prosedur implementasi SGD melibatkan inisialisasi bobot secara acak kemudian melakukan pembaruan bertahap:

1. Untuk setiap contoh pelatihan  $(x_i, y_i)$ :
2. Hitung prediksi  $o_i$  menggunakan bobot saat ini
3. Hitung error  $(y_i - o_i)$
4. Perbarui setiap bobot  $w_k$  dengan:  $w_k = w_k + \eta(y_i - o_i)x_{ik}$  dimana  $\eta$  adalah learning rate

Contoh numerik: Dengan bobot awal  $[0, 0, 0, 0]$ , learning rate 0.3, dan data  $[1, 0, 1]$  label  $-1$ :

- Prediksi awal: 0
- Error:  $-1 - 0 = -1$
- Pembaruan bobot:  $w_0 = 0 + 0.3*(-1)*1 = -0.3$   $w_1 = 0 + 0.3*(-1)*1 = -0.3$   $w_3 = 0 + 0.3*(-1)*1 = -0.3$

Pembaruan langsung ini meningkatkan kecepatan adaptasi model terhadap pola data baru.

## Mini-batch SGD sebagai Hybrid Approach

Mini-batch SGD menyeimbangkan efisiensi SGD dan stabilitas Batch GD dengan membagi data menjadi subgroup berukuran tertentu (biasanya 32-256 sampel). Setiap subgroup diproses secara paralel sebelum pembaruan bobot dilakukan. Pendekatan ini memberikan: - **Estimasi gradien lebih stabil** dibanding SGD murni karena menggunakan beberapa sampel - **Efisiensi komputasi** lebih baik dibanding Batch GD penuh - Kompatibilitas dengan akselerasi hardware (GPU/TPU) Jika ukuran mini-batch = 1, maka menjadi SGD murni. Jika ukuran = seluruh dataset, menjadi Batch GD. Implementasi praktis umumnya menggunakan ukuran 64 atau 128 sebagai titik optimal antara kecepatan dan stabilitas.

## Analisis Perbandingan Variant GD

Karakteristik	Batch GD	SGD	Mini-batch SGD
Frekuensi Update	1 per epoch	n_samples per epoch	n_batches per epoch
Konsistensi Gradien	Tinggi	Rendah	Moderate
Kemampuan Generalisasi	Baik	Sangat Baik	Baik
Kebutuhan Memori	Tinggi	Rendah	Moderate
Cocok untuk dataset berukuran kecil-sedang, sedangkan SGD dan mini-batch lebih optimal untuk data besar atau streaming data.			

[!cornell] ##### Summary

**Stochastic Gradient Descent (SGD)** meningkatkan efisiensi pelatihan model dengan melakukan pembaruan bobot per sampel menggunakan rumus  $w_k = w_k + \eta(y_i - o_i)x_{ik}$ , menghasilkan konvergensi lebih cepat untuk data besar. **Mini-batch SGD** menggabungkan keunggulan SGD dan Batch GD melalui pembaruan berbasis subgroup, menyeimbangkan stabilitas dan efisiensi komputasi. Pemilihan varian bergantung pada ukuran data, sumber daya komputasi, dan kebutuhan stabilitas konvergensi.

[!ad-libitum]- Additional Information

## Analisis Matematis Update Rule

Pembaruan bobot SGD merupakan turunan dari prinsip gradient descent dimana gradien error function dihitung sebagai:  $\nabla E(w) = -(y_i - o_i)x_i$  Pembaruan dilakukan ke arah negatif gradien:  $w_{t+1} = w_t - \eta \nabla E(w)$  Untuk fungsi error kuadratik  $E(w) = 1/2(y_i - o_i)^2$ , turunan parsial terhadap  $w_k$  adalah:  $\partial E / \partial w_k = -(y_i - o_i)x_{ik}$

## Konvergensi dan Learning Rate Scheduling

Konvergensi SGD dijamin dengan learning rate yang memenuhi kondisi Robbins-Monro:  $\sum \eta_t = \infty$  dan  $\sum \eta_t^2 < \infty$  Praktik umum menggunakan adaptive learning rate seperti: - Decaying:  $\eta_t = \eta_0 / (1 + \alpha t)$  - AdaGrad:  $\eta_t = \eta_0 / \sqrt{(\sum g_t^2 + \epsilon)}$  - Momentum:  $v_t = \gamma v_{t-1} + \eta g_t$  lalu  $w_{t+1} = w_t - v_t$

## Implementasi Praktis dengan Hyperparameter Tuning

Parameter kritis dalam implementasi: - **Ukuran Mini-batch:** 32-256 untuk kebanyakan kasus - **Learning Rate Awal:** 0.01-0.1 untuk model linear - **Jadwal Pembelajaran:** Step decay (mis. turun 50% tiap 20 epoch) - **Regularisasi:** Penambahan L2/L1 penalty pada update rule Teknik advanced: Nesterov Accelerated Gradient, AMSGrad, Adam

## Studi Kasus: Perbandingan Kinerja pada MNIST

Metode	Akurasi (%)	Waktu Pelatihan (s)
Batch GD	92.1	120
SGD	93.8	45
Mini-batch(64)	94.2	28
Mini-batch(256)	93.5	22

## Self-Exploration Projects

1. Implementasikan SGD dari scratch dengan Python untuk regresi linear pada dataset Boston Housing. Bandingkan konvergensi dengan variasi learning rate (0.001, 0.01, 0.1).
2. Eksperimen mini-batch size pada model CNN sederhana menggunakan TensorFlow/Keras untuk klasifikasi CIFAR-10. Ukur waktu pelatihan dan akurasi untuk batch size 32, 64, 128.

## Tools and Resources

- PyTorch: `torch.optim.SGD` dengan opsi momentum dan weight decay
- TensorFlow: `tf.keras.optimizers.SGD`
- Scikit-learn: `SGDClassifier` dan `SGDRegressor`
- Library Visualisasi: Weights & Biases (wandb) untuk tracking eksperimen

## Further Reading

- "Stochastic Gradient Descent Tricks" oleh Léon Bottou (2012)
- "Understanding Machine Learning: From Theory to Algorithms" (Shalev-Shwartz & Ben-David), Bab 14
- Paper: "On the Convergence of Adam and Beyond" (ICLR 2018)
- Tutorial: "An Overview of Gradient Descent Optimization Algorithms" (Sebastian Ruder)

---

type: Note cssclasses: - cornell-notes

---

Back to [[Pembelajaran Mesin IF3270]]

[!cornell] Perceptron Applications in Logic Functions

## Questions/Cues

- Bagaimana perceptron merepresentasikan fungsi logika dasar?
- Apa itu fungsi m-of-n dalam konteks perceptron?
- Mengapa bobot identik digunakan untuk fungsi m-of-n?
- Bagaimana menentukan nilai ambang (threshold) untuk operasi AND?
- Contoh implementasi perceptron untuk fungsi OR

## Reference Points

- Lecture\_01\_DFS.pptx (Slides 9-12)
- Machine Learning Mitchell (Chapter 4.4)
- Raschka (2022) Machine Learning with PyTorch and Scikit-Learn

## Representasi Fungsi Logika dengan Perceptron

Perceptron merupakan model linear yang mampu merepresentasikan berbagai fungsi logika dasar melalui konfigurasi bobot dan nilai ambang (threshold) yang tepat. Strukturnya terdiri dari unit input, bobot, fungsi aktivasi tangga (step function), dan output biner ( $\pm 1$  atau 0/1). **Fungsi logika dasar** seperti AND, OR, NAND dapat diimplementasikan dengan menentukan kombinasi bobot ( $w$ ) dan bias ( $w_0$ ) yang sesuai. Contoh implementasi fungsi OR: - Input:  $x_1, x_2 \in \{0,1\}$  atau  $\{-1,1\}$  - Bobot:  $w_1 = 2, w_2 = 2$  - Bias:  $w_0 = -1$  - Fungsi aktivasi:  $\text{sign}(2x_1 + 2x_2 - 1)$  - Output akan bernilai +1 jika minimal satu input bernilai 1

## Konsep Fungsi m-of-n

Fungsi m-of-n merupakan kelas khusus fungsi logika dimana output bernilai benar (+1) jika minimal  $m$  dari  $n$  input bernilai benar. **Kekuatan perceptron** terletak pada kemampuannya merepresentasikan fungsi ini secara efisien melalui konfigurasi parameter terstandarisasi: 1. Setiap bobot input diatur ke nilai identik (biasanya 0.5 atau 1) 2. Nilai ambang (threshold) dihitung berdasarkan rumus:  $w_0 = -(m - 0.5)$  Contoh kasus AND ( $m = n = 2$ ): - Bobot:  $w_1 = w_2 = 0.5$  - Bias:  $w_0 = -0.8$  - Perhitungan:  $\text{sign}(0.5x_1 + 0.5x_2 - 0.8)$  - Sistem akan mengaktifkan output (+1) hanya jika kedua input bernilai 1

## Implementasi Praktis Fungsi AND

Implementasi fungsi AND menggunakan perceptron menunjukkan bagaimana model linear membentuk **bidang keputusan** (decision boundary) di ruang fitur:  $| x_1 | x_2 | \Sigma (0.5x_1 + 0.5x_2 - 0.8) | \text{Output} |$   

1	1	0.2	1
1	-1	-0.8	-1
-1	1	-0.8	-1
-1	-1	-1.8	-1

 Representasi visual menunjukkan garis pemisah  $0.5x_1 + 0.5x_2 = 0.8$  yang membatasi daerah positif dan negatif secara sempurna.

## Desain Bobot untuk Fungsi Logika

Strategi umum dalam mendesain perceptron untuk fungsi logika: 1. **Identifikasi pola input-output** dari tabel kebenaran 2. **Tentukan persamaan linear** yang memisahkan kelas positif-negatif 3. **Konversi ke bentuk bobot** dengan memperhatikan skala input 4. **Uji coba** terhadap semua kombinasi input Untuk fungsi OR dengan input biner ( $\pm 1$ ): - Persamaan:  $\text{sign}(2x_1 + 2x_2 - 1)$  - Rasional: Output positif jika minimal satu input = +1 - Verifikasi: - (1,1):  $2(1) + 2(1) - 1 = 3 \rightarrow +1$  - (1,-1):  $2(1) + 2(-1) - 1 = -1 \rightarrow -1$  - (-1,1):  $2(-1) + 2(1) - 1 = -1 \rightarrow -1$  - (-1,-1):  $2(-1) + 2(-1) - 1 = -5 \rightarrow -1$

**Perceptron** mampu merepresentasikan berbagai **fungsi logika** melalui konfigurasi bobot dan bias yang tepat, khususnya untuk fungsi **m-of-n** yang mengaktifkan output ketika minimal  $m$  dari  $n$  input bernilai benar. Implementasi fungsi AND menunjukkan bagaimana **bobot identik** (0.5) dan **bias spesifik** (-0.8) membentuk bidang keputusan linear yang efektif. Konsep ini menjadi fondasi untuk memahami kapabilitas dan **batasan perceptron** dalam menyelesaikan masalah yang secara linear terpisah.

[!ad-libitum]- Additional Information

## Problem XOR dan Batasan Perceptron

Meskipun efektif untuk fungsi AND dan OR, perceptron tunggal **tidak dapat** merepresentasikan fungsi XOR karena memerlukan pemisahan non-linear. Solusi memerlukan jaringan multi-layer dengan hidden layer yang mengkombinasikan beberapa perceptron. Contoh representasi XOR: - Layer 1: Implementasi NAND dan OR - Layer 2: Kombinasi AND dari output layer sebelumnya

## Implementasi dengan Python

```
```python
import numpy as np

class LogicPerceptron:
    def __init__(self, weights, bias):
        self.weights = np.array(weights)
        self.bias = bias

    def predict(self, inputs):
        summation = np.dot(inputs, self.weights) + self.bias
        return 1 if summation >= 0 else -1
```
```

## AND Perceptron

```
and_perceptron = LogicPerceptron(weights=[0.5, 0.5], bias=-0.8)
print(and_perceptron.predict([1, 1])) # Output: 1
print(and_perceptron.predict([1, -1])) # Output: -1
```
```

Fungsi Logika Lainnya

1. **NAND**: Bobot negatif (contoh: $w_1=-0.5$, $w_2=-0.5$, $w_0=0.8$)
2. **NOR**: Kombinasi bobot negatif dan threshold tinggi (contoh: $w_1=-1$, $w_2=-1$, $w_0=0.5$)
3. **Fungsi Mayoritas**: m-of-n dimana $m > n/2$ (contoh: 2-of-3 dengan $w_1=w_2=w_3=1$, $w_0=-1.5$)

Bacaan Lanjut

- "Neural Networks and Deep Learning" Chapter 1 oleh Michael Nielsen
- Paper asli: Rosenblatt, F. (1958). The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain
- Tutorial interaktif: <https://playground.tensorflow.org>

Proyek Eksplorasi Mandiri

1. Bangun perceptron untuk fungsi logika 3-input: $(x_1 \text{ AND } x_2) \text{ OR } x_3$
2. Visualisasi bidang keputusan 3D menggunakan Matplotlib
3. Eksperimen dengan representasi biner $\{0,1\}$ vs bipolar $\{-1,1\}$
4. Implementasi gerbang logika kombinasi (mis. Half-Adder) dengan jaringan perceptron